Assignment 6: Medians and Order Statistics & Elementary Data Structures

Fathiya Adan

University of Cumberlands

Algorithms and Data Structures (MSCS-532-B01)

Michael Solomon

Avril 5, 2026

Introduction

This assignment focused on two major topics in algorithm design and data structures. The first part required the implementation and analysis of algorithms for finding the k-th smallest element in an array, also known as the order statistics problem. Specifically, the assignment required one deterministic algorithm with worst-case linear time and one randomized algorithm with expected linear time. The second part required the implementation and discussion of elementary data structures, including arrays, matrices, stacks, queues, and linked lists. The assignment also required performance analysis and discussion of practical applications.

The purpose of this work was not only to produce working code, but also to understand how these algorithms and data structures behave in theory and in practice. For that reason, both correctness testing and empirical analysis were included.

Part 1: Implementation and Analysis of Selection Algorithms

1.1 Problem Definition

The problem of order statistics asks for the element that would appear at a specific position if the array were sorted. For example, in the array:

[7, 2, 9, 4, 1]

the sorted order is:

[1, 2, 4, 7, 9]

Therefore:

- the 1st smallest element is 1
- the 2nd smallest element is 2
- the 3rd smallest element is 4

Instead of sorting the entire array, selection algorithms try to find the desired element more efficiently.

1.2 Randomized Quickselect

Code approach

The first algorithm implemented was **Randomized Quickselect**. The main function validates the input and then calls a recursive helper function. The helper function works by choosing a random

pivot from the array, partitioning the array into three groups, and recursively continuing only on the partition that may contain the desired element.

The code used three partitions:

- lows: all elements smaller than the pivot
- pivots: all elements equal to the pivot
- highs: all elements larger than the pivot

This was an important design choice because it allows the algorithm to handle duplicate values correctly.

How the code works

The general logic of the code is:

1. Check whether the array is empty or whether $k$is invalid.
2. Choose a random pivot.
3. Partition the array into lows, pivots, and highs.
4. Compare $k$to the size of these partitions:
    - If $k$is in lows, recurse on lows
    - If $k$falls within the pivot section, return the pivot
    - Otherwise recurse on highs with an adjusted value of k

Why this method was used

The correctness comes from the fact that after partitioning:

- every element in lows is smaller than the pivot
- every element in pivots is equal to the pivot
- every element in highs is larger than the pivot

So once the sizes of those partitions are known, it is possible to determine exactly where the k-th smallest element must be.

Time complexity

- Best case: $O(n)$
- Expected case: $O(n)$
- Worst case: $O(n^2)$

The expected running time is linear because, on average, the random pivot produces reasonably balanced partitions. However, if poor pivots are repeatedly chosen, the recursion can become highly unbalanced, leading to quadratic time.

Space complexity

The implementation uses additional lists for partitioning, so the space overhead is not constant. The recursive nature of the algorithm also contributes stack usage.

1.3 Deterministic Selection: Median of Medians

Code approach

The second algorithm implemented was the deterministic selection algorithm using **Median of Medians**. This algorithm was required because the assignment specifically asked for a deterministic algorithm that runs in worst-case linear time.

The code followed the classical Median of Medians strategy:

1. If the array is small, sort it directly and return the answer.
2. Divide the array into groups of five.
3. Sort each group and extract its median.
4. Recursively find the median of the medians.
5. Use that value as the pivot.
6. Partition the array into lows, pivots, and highs.
7. Recurse only in the relevant partition.

Why groups of five were used

Groups of five are used in the standard deterministic selection algorithm because they give a strong enough guarantee on partition quality. The median of medians is not necessarily the true median of the full array, but it is guaranteed to be “good enough” so that a significant portion of elements can be discarded at each recursive step.

The code

The key difference from Quickselect is pivot selection. Instead of choosing a pivot randomly, the code carefully computes a pivot by:

- breaking the array into sublists of size five
- sorting those small groups

- taking the median from each group
- recursively selecting the median from the list of medians

This pivot is then used for partitioning exactly as in Quickselect.

Why this code is correct

The algorithm still relies on the same partitioning idea as Quickselect, but the pivot is chosen in a principled way. Since the partitioning is correct and the recursive call is made only on the relevant subproblem, the algorithm correctly returns the k-th smallest element.

Time complexity

- Worst case: $O(n)$

This is the major advantage of the deterministic algorithm. Unlike Randomized Quickselect, its running time remains linear even in the worst case.

Why worst-case linear time is guaranteed

The median of medians pivot ensures that the recursive calls do not become too unbalanced. A fixed fraction of elements is always discarded, which leads to a linear recurrence and therefore linear worst-case complexity.

Space complexity

Like the Quickselect implementation, this version also uses additional lists and recursion, so there is extra memory usage beyond the input array itself.

1.4 Explanation of the Testing Code

After implementing both algorithms, correctness tests were added. The code tested two arrays:

1. a normal array with distinct values
2. an array containing duplicates

For each array, the code requested every order statistic from 1 to n. This is a strong correctness check because it verifies that the algorithm works across the entire sorted order, not just for one specific value of k.

Test arrays used

- Distinct values:

[7, 2, 9, 4, 1, 6, 3, 8, 5]

- Duplicates:

[4, 2, 5, 2, 8, 2, 9, 1]

Results obtained

For the first array, both algorithms returned:

1, 2, 3, 4, 5, 6, 7, 8, 9

For the second array, both algorithms returned:

1, 2, 2, 2, 4, 5, 8, 9

These outputs match the sorted versions of the arrays exactly, showing that both implementations were correct.

Interpretation of correctness results

These tests show that:

- both algorithms correctly identify the k-th smallest element
- duplicates are handled correctly
- the deterministic and randomized implementations produce consistent results

That consistency is especially important, because it confirms that the algorithms solve the same problem correctly despite using different pivot strategies.

1.5 Empirical Analysis

The assignment also required empirical comparison of the two algorithms on different input sizes and distributions.

Experimental setup

The empirical analysis code did the following:

- generated arrays of sizes:
    - 100
    - 1000
    - 5000
    - 10000
- tested three distributions:
    - random

    - sorted
    - reverse
- measured average execution time over 5 trials
- selected the median position as the order statistic

Why the average of several trials was used

A single execution time may not be reliable due to randomness and system-level variation. Averaging over several runs gives a fairer and more stable estimate of performance.

Actual results obtained

The following results were observed:

Size	Distribution	Quickselect (s)	Deterministic (s)
100	random	0.000073	0.000073
100	sorted	0.000092	0.000063
100	reverse	0.000042	0.000119
1000	random	0.000433	0.000763
1000	sorted	0.000388	0.000605
1000	reverse	0.000412	0.000627
5000	random	0.002024	0.004282
5000	sorted	0.001676	0.003139
5000	reverse	0.001463	0.003251
10000	random	0.003860	0.009316
10000	sorted	0.003205	0.007322
10000	reverse	0.003586	0.007091

Interpretation of the empirical results

Several important observations can be made.

1. Both algorithms scale approximately linearly

As the input size increases, the running time of both algorithms also increases in a roughly linear way. This supports the theoretical analysis.

For example, Quickselect on random arrays increased from:

- 0.000073 seconds at size 100
- to 0.003860 seconds at size 10000

Likewise, deterministic selection increased from:

- 0.000073 seconds at size 100
- to 0.009316 seconds at size 10000

This trend is consistent with linear-time behavior.

2. Randomized Quickselect is faster in practice

For almost every test case, Quickselect was faster than the deterministic algorithm. This is expected because Quickselect has lower overhead. It simply chooses a pivot and partitions the array, whereas the deterministic algorithm must:

- divide into groups of five
- sort each small group
- build the list of medians
- recursively select the pivot from those medians

These extra operations increase the constant factors of the deterministic algorithm.

3. Deterministic selection is more expensive but more predictable

Although the deterministic version was slower, it provides a worst-case guarantee of $O(n)$. This makes it theoretically stronger, even though it is not as fast in practice on the tested inputs.

4. Input distribution had limited impact

The results for random, sorted, and reverse-sorted arrays were fairly close for both algorithms. This suggests that both algorithms are robust to different input patterns, especially because the implementations rely on pivot-based partitioning rather than complete sorting.

1.6 Part 1 Conclusion

Part 1 showed that both Randomized Quickselect and Deterministic Selection correctly solve the order statistics problem. The testing results confirmed correctness on both distinct and duplicate values, while the empirical results showed that both algorithms scale efficiently.

In practice, Randomized Quickselect was the faster method. However, Deterministic Selection remains important because it guarantees worst-case linear time. This part of the assignment demonstrates a common trade-off in algorithms: a theoretically stronger guarantee often comes at the cost of greater overhead in practice.

Part 2: Elementary Data Structures

2.1 Overview

The second part of the assignment required implementing elementary data structures from scratch in Python, along with a discussion of performance and practical applications.

The implemented data structures were:

- custom array
- custom matrix
- stack
- queue
- singly linked list

Each structure was tested to verify correctness.

2.2 Custom Array

Code explanation

The CustomArray class was implemented using a Python list internally, but the operations were exposed explicitly through methods:

- access(index)
- insert(index, value)
- delete(index)
- display()

This approach makes the data structure behavior clear and keeps the implementation educational.

How the code works

- access(index) returns the element at the requested position after checking bounds

- insert(index, value) inserts a value at the requested index
- delete(index) removes and returns the element at that index
- display() prints the full contents

Results obtained

The array test inserted:

- 10 at index 0
- 20 at index 1
- 30 at index 2
- 15 at index 1

This produced:

[10, 15, 20, 30]

Accessing index 2 returned:

20

Deleting index 1 removed:

15

The final array became:

[10, 20, 30]

Interpretation

These results confirm that insertion at arbitrary positions, element access, and deletion all worked correctly.

Time complexity

- Access: $O(1)$
- Insert: $O(n)$
- Delete: $O(n)$
- Display: $O(n)$

Access is constant time because arrays support direct indexing. Insertions and deletions are linear because elements may need to shift.

2.3 Custom Matrix

Code explanation

The CustomMatrix class was implemented as a 2D list. It supports:

- construction with fixed number of rows and columns
- access to a specific cell
- updating a specific cell
- displaying the full matrix

How the code works

The constructor creates a matrix filled with a default value. The access(row, col) and update(row, col, value) methods use row-column indexing, while display() prints each row.

Results obtained

A 3×3 matrix was created and the following values were updated:

- (0,0) = 5
- (1,1) = 10
- (2,2) = 15

The output was:

```
[5, 0, 0]
[0, 10, 0]
[0, 0, 15]
```

Accessing element (1,1) returned:

10

Interpretation

These results confirm that matrix update and access worked correctly and that the 2D structure was represented properly.

Time complexity

- Access: $O(1)$
- Update: $O(1)$
- Display: $O(r \times c)$

2.4 Stack

Code explanation

The stack was implemented using a Python list and follows the **Last-In, First-Out (LIFO)** principle. The methods included:

- push(value)
- pop()
- peek()
- is_empty()
- display()

How the code works

- push() appends a new element to the end of the list
- pop() removes the last element
- peek() returns the last element without removing it

Results obtained

The stack test pushed:

- 10
- 20
- 30

The output showed:

Stack contents: [10, 20, 30]
Top element: 30
Popped value: 30
Stack contents: [10, 20]

Interpretation

This confirms correct LIFO behavior, because the last value added, 30, was the first one removed.

Time complexity

- Push: $O(1)$

- Pop: $O(1)$
- Peek: $O(1)$
- Is empty: $O(1)$

Practical applications

Stacks are used in:

- function call management
- undo/redo systems
- expression evaluation
- depth-first search

2.5 Queue

The queue was implemented using a Python list and follows the **First-In, First-Out (FIFO)** principle. The methods included:

- enqueue(value)
- dequeue()
- front()
- is_empty()
- display()

How the code works

- enqueue() appends a value to the rear of the queue
- dequeue() removes the first element
- front() returns the first element without removing it

Results obtained

The queue test inserted:

- 10
- 20
- 30

The output showed:

Queue contents: [10, 20, 30]
Front element: 10
Dequeued value: 10
Queue contents: [20, 30]

Interpretation

This confirms correct FIFO behavior because the first value inserted, 10, was the first value removed.

Time complexity

- Enqueue: $O(1)$
- Dequeue: $O(n)$
- Front: $O(1)$

The dequeue operation is linear because removing the first element from a Python list shifts the remaining elements.

Practical applications

Queues are used in:

- task scheduling
- printer spooling
- network packet processing
- breadth-first search

2.6 Singly Linked List

Code explanation

The singly linked list was implemented using two classes:

- Node, which stores data and a pointer to the next node
- SinglyLinkedList, which manages the list

The linked list supports:

- insertion at beginning
- insertion at end

- deletion by value
- search
- display

How the code works

- insert_at_beginning(value) creates a new node and makes it the new head
- insert_at_end(value) traverses to the end and appends a new node
- delete(value) removes the first node containing the target value
- search(value) scans the list for a matching value
- display() traverses the list and prints all node values

Results obtained

The linked list test produced:

Linked List: [5, 10, 20, 30]
Search 20: True
Search 100: False
After deleting 20:
Linked List: [5, 10, 30]
After deleting 5:
Linked List: [10, 30]

Interpretation

These results confirm that:

- insertion at the beginning worked
- insertion at the end worked
- searching worked correctly
- deleting both an internal node and the head node worked correctly

Time complexity

- Insert at beginning: $O(1)$
- Insert at end: $O(n)$
- Search: $O(n)$
- Delete: $O(n)$

- Display: $O(n)$

Practical applications

Linked lists are useful in:

- dynamic memory management
- implementing stacks and queues
- adjacency-list representations of graphs
- systems where frequent insertions/deletions are needed

2.7 Trade-Off Discussion

Arrays vs linked lists

Arrays are better when:

- fast random access is needed
- memory layout is simple
- indexing is frequent

Linked lists are better when:

- frequent insertion and deletion are needed
- the size changes dynamically
- direct indexed access is not important

The major trade-off is:

- arrays provide $O(1)$access but expensive insertions/deletions
- linked lists support easier structural modification but slower access

Stacks vs queues

Stacks and queues are both linear data structures, but their removal order differs:

- stack: LIFO
- queue: FIFO

This makes them useful for different types of problems.

Why implementation choice matters

The queue implementation in this assignment used a Python list, so dequeue was $O(n)$. If a linked-list-based queue had been used, dequeue from the front could be reduced to $O(1)$. This is a good example of how implementation details affect performance.

2.8 Part 2 Conclusion

Part 2 demonstrated how elementary data structures behave through direct implementation and testing. Arrays and matrices provided fast indexed access, stacks demonstrated LIFO processing, queues demonstrated FIFO processing, and linked lists showed how dynamic node-based storage works.

The results confirmed that all structures functioned correctly, while the complexity analysis highlighted that no single structure is best for every situation. The choice depends on the operations that need to be performed most efficiently.

Overall, the assignment demonstrated that theoretical complexity analysis and real implementation behavior must both be considered when choosing algorithms and data structures for real applications.

References

Cormen, T. H., Charles E. Leiserson, Ronald L. Rivest, & Clifford Stein. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., & Tarjan, R. E. (1973). Time bounds for selection. *Journal of Computer and System Sciences, 7*(4), 448–461.

Python Software Foundation. (2024).